



OAuth 2.0 API Documentation

Developer Integration Guide

Version 1.0 — 25 March 2026

Converty OAuth 2.0 API Documentation

Developer guide for integrating your application with the Converty e-commerce platform.

Table of Contents

- [Overview](#)
 - [Getting Started](#)
 - [Authentication Flow](#)
 - [Step 1: Redirect to Authorization](#)
 - [Step 2: User Grants Permission](#)
 - [Step 3: Exchange Code for Tokens](#)
 - [Step 4: Use the Access Token](#)
 - [Step 5: Refresh an Expired Token](#)
 - [Scopes](#)
 - [API Reference](#)
 - [Stores](#)
 - [Products](#)
 - [Orders](#)
 - [Webhooks](#)
 - [Rate Limits](#)
 - [Error Handling](#)
 - [Implementation Guide](#)
-

Overview

Converty uses **OAuth 2.0 Authorization Code** grant to allow third-party applications to access store data on behalf of Converty sellers. There are no static API keys — every integration is tied to a specific seller's store through the OAuth consent flow.

Base URL:

```
https://partner.converty.shop
```

Token lifetimes:

TOKEN	LIFETIME
Authorization code	5 minutes
Access token	15 days
Refresh token	1 year

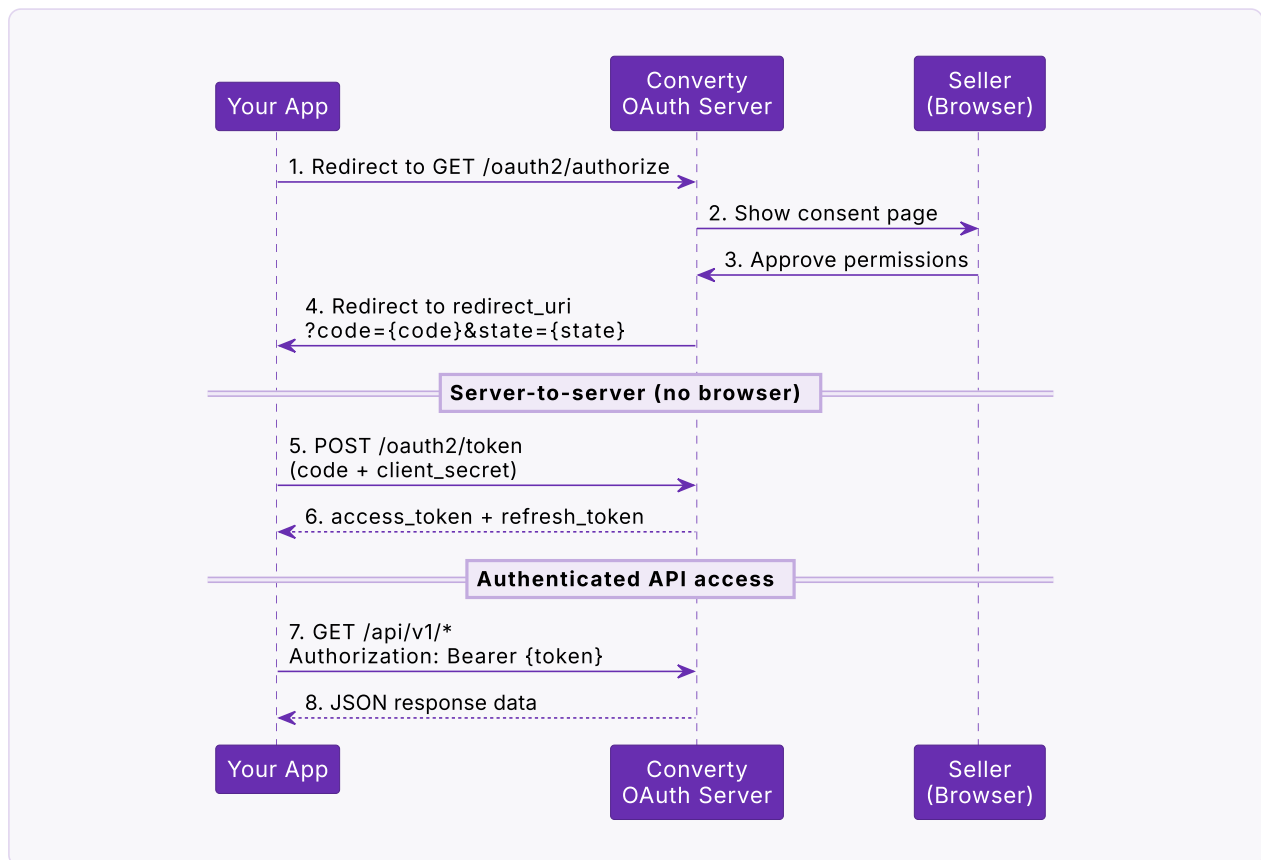
Getting Started

Before you can integrate, you need credentials from Convery:

1. **Contact Convery** and describe your application and the scopes you need.
2. Convery will register an OAuth Client for you and provide:
 - `client_id` — your application's unique identifier
 - `client_secret` — a secret key (keep this secure, never expose it client-side)
3. Provide Convery with your **redirect URIs** — the URLs where users will be sent after authorizing your app.

Once you have your credentials, you can implement the authorization flow.

Authentication Flow



Step 1: Redirect to Authorization

Redirect the seller's browser to the Convery authorization page:

```
GET https://partner.convery.shop/oauth2/authorize
```

Query parameters:

PARAMETER	REQUIRED	DESCRIPTION
<code>response_type</code>	Yes	Must be <code>code</code>
<code>client_id</code>	Yes	Your OAuth client ID
<code>redirect_uri</code>	Yes	One of your registered redirect URIs
<code>scope</code>	Yes	Space-separated list of <code>scopes</code>
<code>state</code>	Recommended	An opaque string for CSRF protection. Returned unchanged in the callback.

Example:

```
https://partner.converty.shop/oauth2/authorize?
  response_type=code&
  client_id=6507a1b2c3d4e5f678901234&
  redirect_uri=https://yourapp.com/callback&
  scope=read-products read-orders read-stores&
  state=xyzABC123
```

The seller must be logged in to their Converty admin dashboard. They will see a consent screen showing your app name and the permissions you are requesting.

Possible errors at this step:

HTTP STATUS	MESSAGE	CAUSE
400	Invalid client	The <code>client_id</code> does not exist
400	Invalid redirect URI	The <code>redirect_uri</code> doesn't match any registered URI
400	Invalid scope	One or more requested scopes are not allowed for your client

Step 2: User Grants Permission

When the seller approves, Converty redirects the browser to your `redirect_uri` with an authorization code:

```
https://yourapp.com/callback?code=a1b2c3d4-e5f6-7890-abcd-ef1234567890&state=xyzABC123
```

Callback parameters:

PARAMETER	DESCRIPTION
<code>code</code>	The authorization code. Valid for 5 minutes . Single-use.
<code>state</code>	The same <code>state</code> value you sent in Step 1. Verify this matches to prevent CSRF attacks.

Important: Always validate that the returned `state` matches the one you sent. If it doesn't match, reject the request.

Step 3: Exchange Code for Tokens

Exchange the authorization code for an access token and refresh token by making a server-side request.

```
POST https://partner.converty.shop/oauth2/token
Content-Type: application/x-www-form-urlencoded
```

Important: The `Content-Type` must be `application/x-www-form-urlencoded`. JSON bodies are not accepted at this endpoint.

Body parameters:

PARAMETER	REQUIRED	DESCRIPTION
<code>grant_type</code>	Yes	Must be <code>authorization_code</code>
<code>code</code>	Yes	The authorization code from Step 2
<code>client_id</code>	Yes	Your OAuth client ID
<code>client_secret</code>	Yes	Your OAuth client secret

Example request:

```
curl -X POST https://partner.converty.shop/oauth2/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=authorization_code&code=a1b2c3d4-e5f6-7890-abcd-ef1234567890&client_id=6507a1b2c3d4e5f678901234&client_secret=f47ac10b-58cc-4372-a567-0e02b2c3d479"
```

Success response (200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "Bearer",
  "expires_in": 1295999
}
```

FIELD	DESCRIPTION
<code>access_token</code>	JWT token used to authenticate API requests. Valid for 15 days.
<code>refresh_token</code>	JWT token used to obtain new access tokens. Valid for 1 year.
<code>token_type</code>	Always <code>Bearer</code>
<code>expires_in</code>	Token lifetime in seconds (~15 days)

Error responses:

HTTP STATUS	MESSAGE	CAUSE
400	<code>Invalid client credentials</code>	<code>client_id</code> or <code>client_secret</code> is wrong
400	<code>Invalid authorization code</code>	Code is expired, already used, or invalid
400	<code>Invalid client id</code>	The <code>client_id</code> doesn't match the one used in authorization
400	<code>Unsupported grant type</code>	<code>grant_type</code> is not <code>authorization_code</code> or <code>refresh_token</code>

Security: Always exchange the code server-side. Never expose your `client_secret` in frontend code.

Step 4: Use the Access Token

Include the access token in the `Authorization` header of every API request:

```
curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs...\" \
https://partner.converty.shop/api/v1/stores/me
```

The access token is scoped to a specific seller's store. All API calls return data belonging to that store only.

Step 5: Refresh an Expired Token

When the access token expires, use the refresh token to obtain a new one without requiring the seller to re-authorize.

```
POST https://partner.converty.shop/oauth2/token
Content-Type: application/x-www-form-urlencoded
```

Body parameters:

PARAMETER	REQUIRED	DESCRIPTION
<code>grant_type</code>	Yes	Must be <code>refresh_token</code>
<code>refresh_token</code>	Yes	The refresh token from the token response
<code>client_id</code>	Yes	Your OAuth client ID
<code>client_secret</code>	Yes	Your OAuth client secret

Example request:

```
curl -X POST https://partner.converty.shop/oauth2/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-d
"grant_type=refresh_token&refresh_token=eyJhbGciOiJIUzI1NiIs...&client_id=6507a1b2c3d4e5f678901234
&client_secret=f47ac10b-58cc-4372-a567-0e02b2c3d479"
```

Success response (200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 1295999
}
```

Note: If the refresh token itself has expired, a new refresh token will be returned in the response. Always store the latest `refresh_token` from each response, as it may replace the previous one.

Error responses:

HTTP STATUS	MESSAGE	CAUSE
400	Refresh token is required	Missing <code>refresh_token</code> parameter
400	Invalid refresh token	Token is malformed, expired, or doesn't match <code>client_id</code>

Scopes

Scopes define what your application can access. Request only the scopes you need.

SCOPE	DESCRIPTION
<code>read-products</code>	Read products from the store
<code>create-products</code>	Create new products
<code>update-products</code>	Update existing products
<code>read-orders</code>	Read orders from the store
<code>create-orders</code>	Create new orders
<code>update-orders</code>	Update existing orders
<code>read-stores</code>	Read store information
<code>read-hooks</code>	List registered webhooks
<code>create-hooks</code>	Register new webhooks
<code>delete-hooks</code>	Remove webhooks

Request scopes as a space-separated string in the authorization URL:

```
scope=read-products read-orders read-stores
```

If your app tries to call an endpoint without the required scope, the API returns:

```
{
  "success": false,
  "message": "Insufficient permissions: Scopes needed read-products"
}
```

Note: If Convery updates the scopes assigned to your client, all existing access tokens are immediately invalidated. Users will need to re-authorize your app.

API Reference

Base URL: `https://partner.convery.shop/api/v1`

All endpoints require the `Authorization: Bearer {access_token}` header.

Stores

Get Store Info

Returns information about the authenticated seller's store.

```
GET /api/v1/stores/me
```

Required scope: `read-stores`

Example request:

```
curl -H "Authorization: Bearer {access_token}" \
https://partner.convery.shop/api/v1/stores/me
```

Response (200):

```
{
  "success": true,
  "data": {
    "user": {
      "_id": "6507a1b2c3d4e5f678901234",
      "email": "seller@example.com",
      "firstname": "John",
      "lastname": "Doe"
    },
    "_id": "6508b2c3d4e5f67890123456",
    "name": "My Store",
    "slug": "my-store",
    "domain": "https://my-store.convery.shop",
    "logo": "https://cdn.convery.shop/stores/logo.png",
    "title": "My Store - Best Products",
    "currency": "DZD",
    "country": "DZ"
  }
}
```

This endpoint is cached for 12 hours on the server side.

Products

List Products

Retrieve products from the store with pagination.

```
GET /api/v1/products
```

Required scope: `read-products`

Query parameters:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
<code>page</code>	number	1	Page number
<code>limit</code>	number	10	Items per page (max 200)
<code>search</code>	string	—	Search by product name (alphanumeric and spaces only)

Example request:

```
curl -H "Authorization: Bearer {access_token}" \
  "https://partner.converty.shop/api/v1/products?page=1&limit=20"
```

Response (200):

```

{
  "success": true,
  "message": "Fetched 20 products!",
  "count": 150,
  "data": [
    {
      "_id": "6509c3d4e5f6789012345678",
      "name": "Classic T-Shirt",
      "sku": "TSH-001",
      "description": "A comfortable cotton t-shirt",
      "images": [
        { "sm": "https://cdn.../sm.webp", "md": "https://cdn.../md.webp", "lg":
"https://cdn.../lg.webp" }
      ],
      "price": 2500,
      "comparePrice": 3000,
      "cost": 1200,
      "deliveryPrice": 500,
      "deliveryCost": 300,
      "trackStock": true,
      "stock": 45,
      "status": "shown",
      "options": [
        {
          "id": "option-96z2jh7h9",
          "name": "Size",
          "type": "text",
          "values": [
            { "id": "option-value-qckspbp9r", "value": "S" },
            { "id": "option-value-8r7rww3m1", "value": "M" },
            { "id": "option-value-glaanox02", "value": "L" },
          ],
        }
      ],
      "newVariants": [
        {
          "id": "variant-s6m2r0zwu",
          "selectedValues": ["S"],
          "default": false,
          "sku": "TSH-001-S",
          "price": 2500,
          "comparePrice": 0,
          "cost": 16,
          "deliveryPrice": 7,
          "stock": {
            "outOfStock": false,
            "quantity": 10,
            "incoming": 0,
            "alertOn": 0,
            "continueSellingWhenOutOfStock": true
          }
        },
        {
          "id": "variant-vyvsncsz6",

```

```

    "selectedValues": ["M"],
    "default": false,
    "sku": "TSH-001-M",
    "price": 2500,
    "comparePrice": 0,
    "cost": 16,
    "deliveryPrice": 7,
    "stock": {
      "outOfStock": false,
      "quantity": 15,
      "incoming": 0,
      "alertOn": 0,
      "continueSellingWhenOutOfStock": true
    },
  },
  {
    "id": "variant-lilb6ka67",
    "selectedValues": ["L"],
    "default": false,
    "sku": "TSH-001-L",
    "price": 2500,
    "comparePrice": 0,
    "cost": 16,
    "deliveryPrice": 7,
    "stock": {
      "outOfStock": false,
      "quantity": 12,
      "incoming": 0,
      "alertOn": 0,
      "continueSellingWhenOutOfStock": true
    },
  }
],
"bundles": {
  "discountType": "fixed",
  "options": [
    {
      "id": "bundle-6bepn414e",
      "name": "buy 1",
      "label": "Get 0% off",
      "quantity": 1,
      "default": true,
      "image": "https://cdn.../sm.webp",
      "badgeColor": "blue",
      "price": 50,
      "comparePrice": 50,
      "deliveryPrice": 7,
    },
    {
      "id": "bundle-ncqlb3zue",
      "name": "buy 2",
      "label": "Get 25% off",
      "quantity": 2,
      "default": false,
      "image": "https://cdn.../sm.webp",
    }
  ]
}

```

```

        "badgeColor": "green",
        "price": 75,
        "comparePrice": 100,
        "deliveryPrice": 7,
      },
      {
        "id": "bundle-0irag5i9s",
        "name": "buy 3",
        "label": "Get 1 for free + free delivery",
        "quantity": 3,
        "default": false,
        "image": "https://cdn.../sm.webp",
        "badgeColor": "red",
        "price": 100,
        "comparePrice": 150,
        "deliveryPrice": 0,
      }
    ]
  },
  "categories": ["650abc123def456789012345"],
  "createdAt": "2024-01-15T10:30:00.000Z",
  "updatedAt": "2024-03-20T14:45:00.000Z"
}
]
}

```

Create Product

Create a new product in the store.

POST /api/v1/products

Required scope: `create-products`

Request body (JSON):

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>name</code>	string	Yes	Product name
<code>price</code>	number	Yes	Product price
<code>deliveryPrice</code>	number	Yes	Delivery price charged to the customer
<code>cost</code>	number	Yes	Product cost (visible only to seller)
<code>deliveryCost</code>	number	Yes	Actual delivery cost
<code>images</code>	array	Yes	At least one image. Each object: <code>{ "src": "url", "type": "image" }</code>
<code>sku</code>	string	No	SKU code
<code>description</code>	string	No	Product description
<code>comparePrice</code>	number	No	Comparison/original price
<code>trackStock</code>	boolean	No	Enable stock tracking
<code>stock</code>	number	No	Stock quantity
<code>status</code>	string	No	One of: <code>shown</code> , <code>hidden</code> , <code>outOfStock</code> , <code>linkOnly</code> , <code>landingPageOnly</code>
<code>options</code>	array	No	Variant definitions
<code>newVariants</code>	object	No	Price/stock per variant combination (required if <code>options</code> is provided)
<code>bundles</code>	object	No	Bundled pricing and discount options (see below). If provided, must be an object with a <code>discountType</code> and an <code>options</code> array.

Bundles object (optional):

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>discountType</code>	string	Yes	Type of bundle discount. Usually <code>"fixed"</code> .
<code>options</code>	array	Yes	List of bundle options. Each bundle option is an object with the following fields:
(bundle option fields)			
<code>id</code>	string	No	ID for the bundle option.
<code>name</code>	string	Yes	Name or label for the bundle (e.g. "buy 2")
<code>label</code>	string	Yes	Description or marketing label (e.g. "Get 25% off")
<code>quantity</code>	number	Yes	Required cart quantity for the bundle
<code>default</code>	boolean	No	Marks if this bundle is the default option
<code>image</code>	string	No	Image URL for the bundle
<code>badgeColor</code>	string	No	Badge or highlight color. Must be one of: <code>"red"</code> , <code>"green"</code> , or <code>"blue"</code>
<code>price</code>	number	Yes	Bundle price
<code>comparePrice</code>	number	No	Original (non-discounted) price
<code>deliveryPrice</code>	number	No	Delivery price for this bundle

Example request:

```
curl -X POST https://partner.converty.shop/api/v1/products \
-H "Authorization: Bearer {access_token}" \
-H "Content-Type: application/json" \
-d '{
  "name": "Leather Wallet",
  "price": 3500,
  "deliveryPrice": 400,
  "cost": 1800,
  "deliveryCost": 250,
  "images": [{ "src": "https://example.com/wallet.jpg", "type": "image" }],
  "sku": "WLT-001",
  "description": "Genuine leather wallet",
  "trackStock": true,
  "stock": 100,
  "status": "shown"
}'
```

Response (200):

```
{
  "success": true,
  "data": {
    "_id": "650def456789012345678901",
    "reference": 1,
    "name": "Leather Wallet",
    "price": 3500,
    "images": [
      { "sm": "https://cdn.../sm.webp", "md": "https://cdn.../md.webp", "lg":
"https://cdn.../lg.webp" }
    ],
    "createdAt": "2024-03-20T14:45:00.000Z",
    "updatedAt": "2024-03-20T14:45:00.000Z"
  }
}
```

Note: Images are automatically processed by Convertly. The URLs in the response will be CDN URLs in **sm**, **md**, and **lg** sizes, regardless of the original **src** you provided.

Update Product

Update an existing product.

```
PATCH /api/v1/products/:id
```

Required scope: `update-products`

URL parameters:

PARAMETER	DESCRIPTION
<code>id</code>	The product <code>_id</code>

All body fields are optional — send only the fields you want to update.

Example request:

```
curl -X PATCH https://partner.convertly.shop/api/v1/products/6509c3d4e5f6789012345678 \
-H "Authorization: Bearer {access_token}" \
-H "Content-Type: application/json" \
-d '{
  "price": 2800,
  "stock": 30,
  "status": "shown"
}'
```

Response (200):

```
{
  "success": true,
  "data": {
    "_id": "6509c3d4e5f6789012345678",
    "name": "Classic T-Shirt",
    "price": 2800,
    "stock": 30,
    "status": "shown",
    "updatedAt": "2024-03-21T09:00:00.000Z"
  }
}
```

Orders

List Orders

Retrieve orders from the store with pagination.

```
GET /api/v1/orders
```

Required scope: `read-orders`

Query parameters:

PARAMETER	TYPE	DEFAULT	DESCRIPTION
<code>page</code>	number	1	Page number
<code>limit</code>	number	10	Items per page (max 200)

Example request:

```
curl -H "Authorization: Bearer {access_token}" \
  "https://partner.converty.shop/api/v1/orders?page=1&limit=25"
```

Response (200):

```

{
  "success": true,
  "count": 530,
  "message": "Fetched 25 order(s) of page 1!",
  "data": [
    {
      "_id": "650fed789012345678901234",
      "reference": 1042,
      "status": "confirmed",
      "paymentStatus": "unpaid",
      "customer": {
        "name": "Ahmed Ben Ali",
        "phone": "0555123456",
        "phone2": "",
        "email": "",
        "address": "Rue Didouche Mourad",
        "city": "Algiers",
        "town": "Bab El Oued",
        "note": ""
      },
    },
    {
      "cart": [
        {
          "quantity": 2,
          "product": {
            "_id": "6509c3d4e5f6789012345678",
            "reference": "PRD-001",
            "name": "Classic T-Shirt",
            "images": [{ "sm": "...", "md": "...", "lg": "..."}],
            "sku": "TSH-001",
            "price": 2500,
            "deliveryPrice": 500,
            "deliveryCost": 300,
            "cost": 1200
          },
          "selectedVariants": [
            { "name": "Size", "type": "text", "value": "M" }
          ],
          "selectedVariantsId": "variant-vyvsnscsz6",
          "selectedBundleOptionId": "bundle-ncqlb3zue",
          "pricePerUnit": 2500
        }
      ],
      "total": {
        "deliveryPrice": 500,
        "deliveryCost": 300,
        "totalPrice": 5500
      },
      "deliveryCompany": "shipper",
      "barcode": "SHIP-123456",
      "history": [
        {
          "status": "pending",
          "timestamp": 1710900000000,
          "actionTaker": "system"
        }
      ]
    }
  ]
}

```

```
    },
    {
      "status": "confirmed",
      "timestamp": 1710950000000,
      "actionTaker": "seller"
    }
  ],
  "exchange": false,
  "attempt": 0,
  "archived": false,
  "createdAt": "2024-03-20T08:00:00.000Z",
  "updatedAt": "2024-03-20T14:00:00.000Z"
}
]
```

Create Order

Create a new order in the store.

POST /api/v1/orders

Required scope: `create-orders`

Request body (JSON):

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>customer</code>	object	Yes	Customer information (see below)
<code>cart</code>	array	Yes	Array of cart items (non-empty)
<code>total</code>	object	Yes	Order totals (see below)
<code>status</code>	string	No	Initial status: <code>pending</code> (default), <code>confirmed</code> , <code>exchange</code> , <code>packed</code>
<code>exchange</code>	boolean	No	Whether this is an exchange order (default: <code>false</code>)
<code>deliveryCompany</code>	string	No	Delivery provider: <code>personal</code> , <code>shipper</code> , or <code>ecomness</code>
<code>note</code>	string	No	Order note

Customer object:

FIELD	TYPE	DESCRIPTION
<code>name</code>	string	Customer name
<code>phone</code>	string	Primary phone number
<code>phone2</code>	string	Secondary phone number
<code>email</code>	string	Email address
<code>address</code>	string	Delivery address
<code>city</code>	string	City
<code>town</code>	string	Town/district
<code>note</code>	string	Additional notes

Cart item object:

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>product</code>	string	Yes	Product <code>_id</code>
<code>quantity</code>	number	Yes	Quantity (positive integer)
<code>pricePerUnit</code>	number	Yes	Price per unit (≥ 0)
<code>selectedVariants</code>	array	No	Selected variant values: <code>[{ "name": "Size", "type": "text", "value": "M" }]</code>
<code>selectedVariantsId</code>	string	No	The ID of the selected variant combination, if applicable (e.g. <code>"variant-vyvsncsz6"</code>)
<code>selectedBundleOptionId</code>	string	No	The ID of the selected bundle option, if a bundle is chosen

Total object:

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>totalPrice</code>	number	Yes	Total order price (≥ 0)
<code>deliveryPrice</code>	number	No	Delivery price
<code>deliveryCost</code>	number	No	Delivery cost

Example request:

```
curl -X POST https://partner.converty.shop/api/v1/orders \
-H "Authorization: Bearer {access_token}" \
-H "Content-Type: application/json" \
-d '{
  "customer": {
    "name": "Karim Mansouri",
    "phone": "0555987654",
    "address": "123 Boulevard Mohamed V",
    "city": "Oran",
    "town": "Centre"
  },
  "cart": [
    {
      "product": "6509c3d4e5f6789012345678",
      "quantity": 1,
      "pricePerUnit": 2500,
      "selectedVariants": [
        { "name": "Size", "type": "text", "value": "L" }
      ],
      "selectedVariantsId": "variant-lilb6ka67",
      "selectedBundleOptionId": "bundle-ncqlb3zue",
    }
  ],
  "total": {
    "totalPrice": 3000,
    "deliveryPrice": 500,
    "deliveryCost": 300
  },
  "deliveryCompany": "shipper"
}'
```

Response (200):

```
{
  "success": true,
  "data": {
    "_id": "6510ab123456789012345678",
    "reference": 1043,
    "status": "pending",
    "paymentStatus": "unpaid",
    "customer": { ... },
    "cart": [ ... ],
    "total": { ... },
    "createdAt": "2024-03-21T10:00:00.000Z"
  }
}
```

Update Order

Update an existing order.

PATCH /api/v1/orders/:id

Required scope: `update-orders`

URL parameters:

PARAMETER	DESCRIPTION
<code>id</code>	The order <code>_id</code>

Request body (JSON):

All fields are optional. The `status` field accepts:

`pending`, `attempt`, `uploaded`, `confirmed`, `exchange`, `rejected`, `in transit`, `delivered`, `returned`

Example request:

```
curl -X PATCH https://partner.converty.shop/api/v1/orders/650fed789012345678901234 \
-H "Authorization: Bearer {access_token}" \
-H "Content-Type: application/json" \
-d '{
  "status": "in transit"
}'
```

Response (200):

```
{
  "success": true,
  "data": {
    "_id": "650fed789012345678901234",
    "reference": 1042,
    "status": "in transit",
    "updatedAt": "2024-03-21T15:30:00.000Z"
  }
}
```

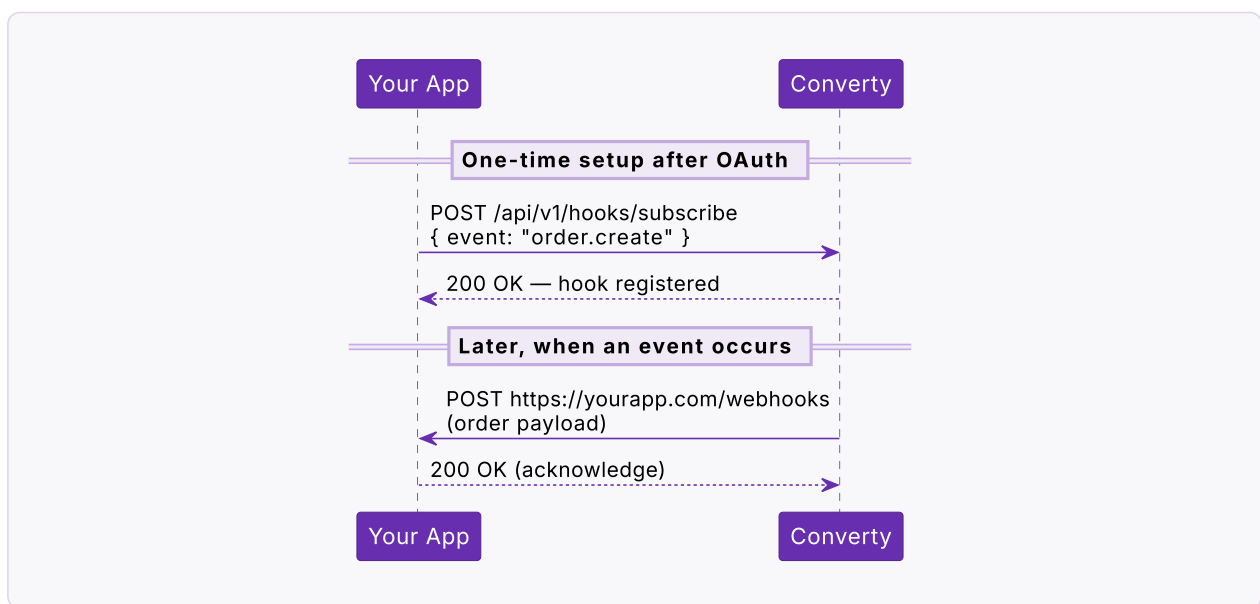
Webhooks

Webhooks allow your app to receive real-time notifications when events happen in the seller's store instead of polling the API.

Supported events:

EVENT	TRIGGERED WHEN
<code>order.create</code>	A new order is placed
<code>order.update</code>	An order is updated (status change, edits, etc.)
<code>product.create</code>	A new product is created
<code>product.update</code>	A product is updated

Constraint: Your webhook `targetUrl` must share the same origin (scheme + host) as one of your registered redirect URIs. For example, if your redirect URI is `https://yourapp.com/callback`, your webhook URL must also be on `https://yourapp.com`.



List Webhooks

```
GET /api/v1/hooks
```

Required scope: `read-hooks`

Example request:

```
curl -H "Authorization: Bearer {access_token}" \
  https://partner.convery.shop/api/v1/hooks
```

Response (200):

```
{
  "success": true,
  "message": "Hooks retrieved successfully",
  "data": [
    {
      "_id": "6511bc234567890123456789",
      "targetUrl": "https://yourapp.com/webhooks/orders",
      "event": "order.create",
      "createdAt": "2024-03-15T12:00:00.000Z",
      "updatedAt": "2024-03-15T12:00:00.000Z"
    },
    {
      "_id": "6511bc234567890123456790",
      "targetUrl": "https://yourapp.com/webhooks/orders",
      "event": "order.update",
      "createdAt": "2024-03-15T12:00:00.000Z",
      "updatedAt": "2024-03-15T12:00:00.000Z"
    }
  ]
}
```

Subscribe to a Webhook

POST /api/v1/hooks/subscribe

Required scope: `create-hooks`

Request body (JSON):

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>targetUrl</code>	string	Yes	A valid URL where Converyty will send event payloads
<code>event</code>	string	Yes	One of: <code>order.create</code> , <code>order.update</code> , <code>product.create</code> , <code>product.update</code>

Example request:

```
curl -X POST https://partner.converyty.shop/api/v1/hooks/subscribe \
-H "Authorization: Bearer {access_token}" \
-H "Content-Type: application/json" \
-d '{
  "targetUrl": "https://yourapp.com/webhooks/orders",
  "event": "order.create"
}'
```

Response (200):

```
{
  "success": true,
  "message": "Hook subscribed successfully",
  "data": {
    "_id": "6511bc234567890123456789",
    "targetUrl": "https://yourapp.com/webhooks/orders",
    "event": "order.create",
    "createdAt": "2024-03-21T16:00:00.000Z",
    "updatedAt": "2024-03-21T16:00:00.000Z"
  }
}
```

Error responses:

HTTP STATUS	MESSAGE	CAUSE
409	Hook already subscribed	A webhook with the same <code>targetUrl</code> + <code>event</code> already exists
403	Invalid target URL	The <code>targetUrl</code> origin doesn't match any registered redirect URI
401	Invalid client	The OAuth client was not found

Unsubscribe from a Webhook

```
DELETE /api/v1/hooks/unsubscribe/:hookId
```

Required scope: `delete-hooks`

URL parameters:

PARAMETER	DESCRIPTION
<code>hookId</code>	The webhook <code>_id</code> from the subscribe response or list endpoint

Example request:

```
curl -X DELETE https://partner.converty.shop/api/v1/hooks/unsubscribe/6511bc234567890123456789 \
-H "Authorization: Bearer {access_token}"
```

Response (200):

```
{
  "success": true,
  "message": "Hook unsubscribed successfully"
}
```

Rate Limits

ENDPOINTS	LIMIT
<code>/oauth2/*</code> (authorization, token)	30 requests per minute
<code>/api/v1/*</code> (all API endpoints)	60 requests per minute

When you exceed the rate limit, the API returns a `429 Too Many Requests` response. Implement exponential backoff in your client.

Error Handling

All error responses follow a consistent format:

```
{
  "success": false,
  "message": "Description of what went wrong"
}
```

Common HTTP status codes:

STATUS	MEANING
200	Success
400	Bad request — invalid parameters or validation error
401	Unauthorized — missing, expired, or invalid token
403	Forbidden — valid token but insufficient scopes
404	Not found — resource doesn't exist
409	Conflict — duplicate resource (e.g., webhook already exists)
429	Rate limited — too many requests
500	Server error — retry later

Token Invalidation

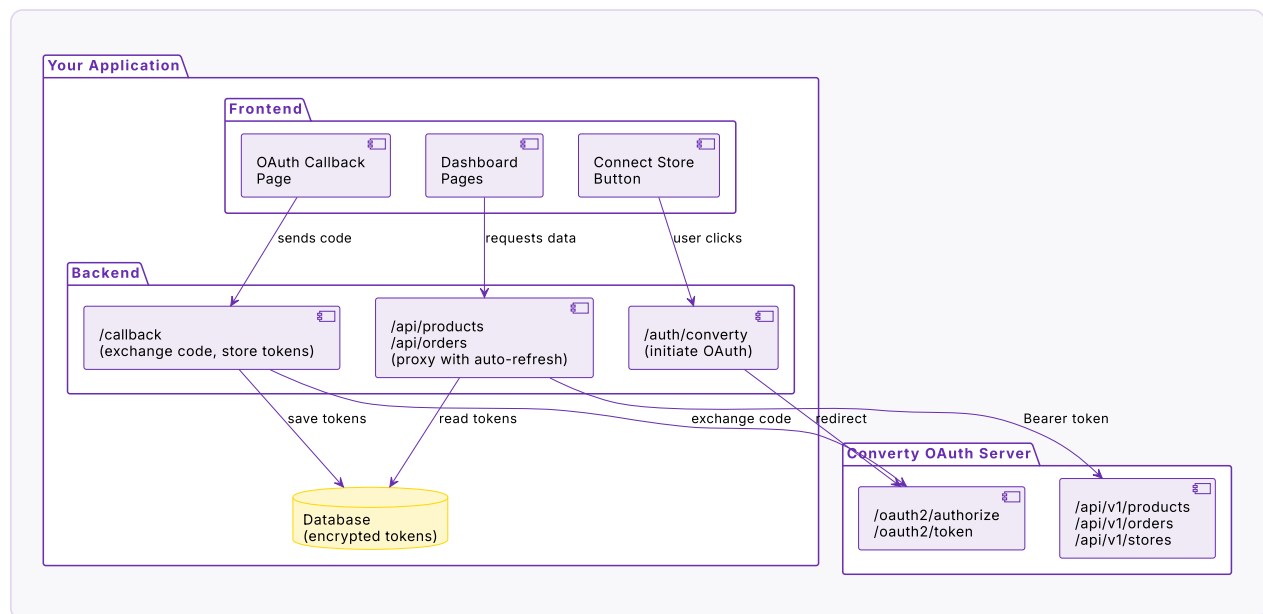
Tokens can be invalidated before their expiration in the following cases:

- **Scope changes:** If Converyt modifies the scopes assigned to your OAuth client, all existing access tokens are immediately invalidated. Your app should handle `401` responses by attempting a token refresh, and if that also fails, prompt the user to re-authorize.

- **Refresh token expired:** If your refresh token has expired (after 1 year), the user must go through the full authorization flow again.

Implementation Guide

Recommended Architecture



Best Practices

1. Store tokens securely on your backend

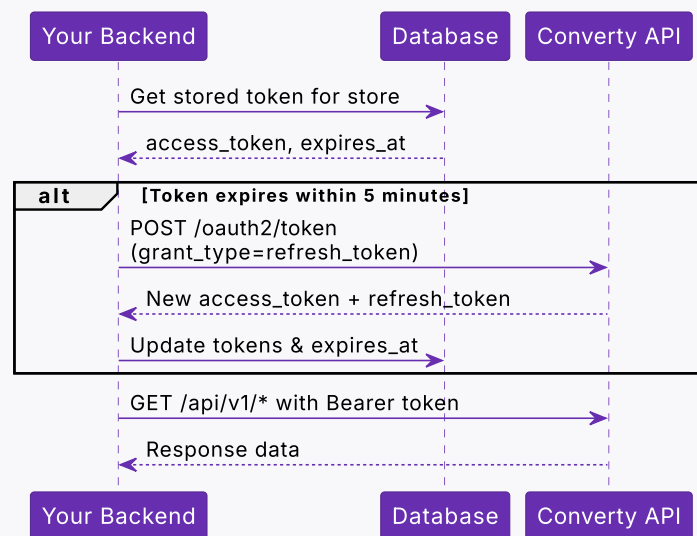
Never store access tokens or refresh tokens in the browser (localStorage, cookies, etc.). Always keep them server-side in your database.

Database record per connected store:

- store_id
- access_token (encrypted at rest)
- refresh_token (encrypted at rest)
- access_token_expires_at
- store_name, store_currency, ...

2. Refresh proactively with a buffer

Don't wait for a `401` to refresh. Check the expiration before each API call and refresh a few minutes early:



```

# Pseudocode
def get_valid_token(store):
    buffer = timedelta(minutes=5)
    if store.access_token_expires_at - buffer ≤ now():
        new_tokens = refresh_access_token(store.refresh_token)
        store.access_token = new_tokens.access_token
        store.refresh_token = new_tokens.refresh_token
        store.access_token_expires_at = now() + timedelta(seconds=new_tokens.expires_in)
        store.save()
    return store.access_token
  
```

3. Handle re-authorization gracefully

If both the access token and refresh token are invalid (e.g., scopes were changed), show the user a prompt to reconnect their store rather than failing silently.

4. Validate the state parameter

Always generate a unique, unpredictable `state` value for each authorization request and validate it in the callback to prevent CSRF attacks:

```

# Pseudocode
def start_auth():
    state = generate_random_string(32)
    save_to_session("oauth_state", state)
    redirect_to_convertly(state=state)

def handle_callback(code, state):
    saved_state = get_from_session("oauth_state")
    if state ≠ saved_state:
        raise Error("Invalid state – possible CSRF attack")
    exchange_code_for_tokens(code)
  
```

5. Subscribe to webhooks for real-time updates

Instead of polling the orders or products endpoints, subscribe to webhooks during the initial setup:

```
# Pseudocode – after successful OAuth callback
def setup_webhooks(access_token):
    events = ["order.create", "order.update", "product.create", "product.update"]
    for event in events:
        post("/api/v1/hooks/subscribe", {
            "targetUrl": "https://yourapp.com/webhooks/convery",
            "event": event
        }, headers={"Authorization": f"Bearer {access_token}"})
```

6. Handle token responses consistently

Always store the latest `refresh_token` from every token response. The server may rotate the refresh token when the current one is close to expiring.

Quick Start Checklist

- ☐ Received `client_id` and `client_secret` from Convery
- ☐ Registered your redirect URI(s) with Convery
- ☐ Built the authorization redirect (Step 1)
- ☐ Built the callback handler to exchange the code for tokens (Step 3)
- ☐ Stored tokens securely in your database
- ☐ Implemented automatic token refresh with a 5-minute buffer
- ☐ Called `GET /api/v1/stores/me` to verify the connection and store the store details
- ☐ Implemented the API calls your app needs (products, orders)
- ☐ Subscribed to relevant webhooks
- ☐ Handled `401` errors by refreshing tokens or prompting re-authorization
- ☐ Tested the full flow end-to-end